1. Demonstrate basic problem-solving using Breadth-First Search on a simple grid.

In [1]:
```
from collections import deque

g = [
    [0, 0, 0],
    [1, 0, 1],
    [0, 0, 0]
]

s = (0, 0)
goal = (2, 2)

def bfs():
    q = deque([s])
    parent = {s: None}

    while q:
        r, c = q.popleft()

        # Goal reached
        if (r, c) == goal:
            path = []

            while True:
                path.append((r, c))

                if parent[(r, c)] is None:
                    break

                r, c = parent[(r, c)]

            return path[::-1]

        # Move in 4 directions
        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
            nr, nc = r + dr, c + dc

            if 0 <= nr < 3 and 0 <= nc < 3:
                if g[nr][nc] == 0 and (nr, nc) not in parent:
                    q.append((nr, nc))
                    parent[(nr, nc)] = (r, c)

    return None

print("Shortest Path:", bfs())
```

```
Shortest Path: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2)]
```

2. Implement Depth-First Search (DFS) on a small graph.

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = set()

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbour in graph[node]:
            dfs(neighbour)

dfs('A')
```

```
A B D E C F
```

3. Solve the Water Jug Problem using Breadth First Search (BFS).

```python
from collections import deque

jug1, jug2, goal = 4,3,2
q = deque([(0,0)])
visited = set()

while q:
    x,y = q.popleft()
    if(x,y) not in visited:
        visited.add((x,y))
        print(x,y)

        if x == goal or y == goal:
            print("Goal reached: ",(x,y))
            break

        transfer = min(x,jug2-y)

        q.extend([
            (jug1,y),(x,jug2),(0,y),(x,0),

            (x-transfer, y+transfer),

            (x+min(y,jug1-x),
            y-min(y,jug1-x)

            )])
```

```
0 0
4 0
0 3
4 3
1 3
3 0
1 0
3 3
0 1
4 2
Goal reached:  (4, 2)
```

4. Implement a Hill Climbing search to find the peak in a numeric dataset.

```python
def hill_climbing_peak(data,start_index = 0):
    n = len(data)
    current = start_index
    while True:

        left = current-1
        right = current+1
        next_index = current

        if left >= 0 and data[left] > data[next_index]:
            next_index = left

        if right<n and data[right]> data[next_index]:
            next_index = right

        if next_index == current:
            return current, data[current]
        current = next_index

data = [1,3,5,7,8,12,9,2]
index, peak = hill_climbing_peak(data)
print("Peak found at index: ",index)
print("Peak value: ",peak)
```

```
Peak found at index:  5
Peak value:  12
```

5. Apply the A* Search algorithm to find the shortest path in a 4x4 grid.

```
start = (0, 0)
goal = (3, 3)

open_list = [start]
g = {start: 0}
parent = {}

while open_list:
    best = open_list[0]

    # Find node with lowest f = g + h
    for node in open_list:
        g1 = g[node]
        h1 = abs(node[0] - goal[0]) + abs(node[1] - goal[1])
        f1 = g1 + h1

        g2 = g[best]
        h2 = abs(best[0] - goal[0]) + abs(best[1] - goal[1])
        f2 = g2 + h2

        if f1 < f2:
            best = node

    current = best
    open_list.remove(current)

    # Goal check
    if current == goal:
        break

    x, y = current

    # Explore neighbors
    for dx, dy in [(0,1), (1,0), (0,-1), (-1,0)]:
        nx = x + dx
        ny = y + dy

        if 0 <= nx < 4 and 0 <= ny < 4 and (nx, ny) != (1,1) and (nx, ny) != (2,3):
            if (nx, ny) not in g:
                g[(nx, ny)] = g[current] + 1
                open_list.append((nx, ny))
                parent[(nx, ny)] = current


# Reconstruct path
path = []
node = goal

while node != start:
    path.append(node)
    node = parent[node]

path.append(start)
path.reverse()

print("Path:", path)
```

```
Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3)]
```

6. Implement the Minmax search algorithm for 2-player games. You may use a game tree with 3 plies.

```
values = [3, 5, 2, 9, 12, 5, 23, 23]

def minimax(depth, node, isMax):

    if depth == 3:
        return values[node]

    if isMax:
        return max(
            minimax(depth + 1, node * 2, False),
            minimax(depth + 1, node * 2 + 1, False)
        )

    else:
        return min(
            minimax(depth + 1, node * 2, True),
            minimax(depth + 1, node * 2 + 1, True)
        )

result = minimax(0, 0, True)

print("Optimal Value:", result)
```

```
Optimal Value: 12
```

---

7. Use constraint propagation to solve a Magic Square puzzle.

```
: from itertools import permutations

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9]

for p in permutations(nums):

    a, b, c, d, e, f, g, h, i = p

    if e != 5:
        continue

    if (
        a + b + c == 15 and
        d + e + f == 15 and
        g + h + i == 15 and
        a + d + g == 15 and
        b + e + h == 15 and
        c + f + i == 15 and
        a + e + i == 15 and
        c + e + g == 15
    ):

        print(a, b, c)
        print(d, e, f)
        print(g, h, i)

        break
```

```
2 7 6
9 5 1
4 3 8
```

---

8. Apply optimization techniques to find the maximum value in a list.

```python
def max_divide_conquer(lst, low, high):

    if low == high:
        return lst[low]

    mid = (low + high) // 2

    left_max = max_divide_conquer(lst, low, mid)
    right_max = max_divide_conquer(lst, mid + 1, high)

    return max(left_max, right_max)

lst = [3, 7, 2, 9, 5]

print("Maximum value: ",max_divide_conquer(lst, 0, len(lst) - 1))
```

```
Maximum value:  9
```

---

9. Represent and evaluate propositional logic expressions.

```python
p = True
q = False
and_result = p and q
or_result = p or q
not_result = not p
implication = (not p) or q
print("p AND q:", and_result)
print("p OR q:", or_result)
print("NOT p:", not_result)
print("p → q:", implication)
```

```
p AND q: False
p OR q: True
NOT p: False
p → q: False
```

10. Implement a basic rule-based expert system for weather classification.

```
def classify_weather(temperature, humidity):

    if temperature > 30 and humidity > 70:
        return "Hot and Humid"

    elif temperature < 20:
        return "Cold"

    else:
        return "Moderate"
print("Weather:", classify_weather(32, 75))
```

```
Weather: Hot and Humid
```

---

11. Implement a basic Al agent with simple decision-making rules.

```
def ai_agent(percept):

    if percept == "hungry":
        return "eat"

    elif percept == "tired":
        return "sleep"

    elif percept == "bored":
        return "play"

    else:
        return "do nothing"


percepts = ["hungry", "tired", "bored", "alert"]

for p in percepts:

    print("Percept:", p, "-> Action:", ai_agent(p))
```

```
Percept: hungry -> Action: eat
Percept: tired -> Action: sleep
Percept: bored -> Action: play
Percept: alert -> Action: do nothing
```

12. Implement a basic Rule-Based Chatbot.

```python
def chatbot(user_input):

    user_input = user_input.lower().strip()

    if "hello" in user_input or "hi" in user_input:
        return "Hi! How can I help you?"

    elif "how are you" in user_input:
        return "I am fine. How about you?"

    elif "your name" in user_input:
        return "I am a simple chatbot."

    elif "fine" in user_input:
        return "Glad to hear that!"

    elif "bye" in user_input:
        return "Goodbye!"

    elif user_input.isalpha():
        return "Nice to meet you, " + user_input.capitalize()

    else:
        return "Sorry, I don't understand."


while True:

    user = input("You: ")

    print("Bot:", chatbot(user))

    if "bye" in user.lower():
        break
```

```
You: hi
Bot: Hi! How can I help you?
You: your name
Bot: I am a simple chatbot.
You: how are you
Bot: I am fine. How about you?
You: fine
Bot: Glad to hear that!
You: bye
Bot: Goodbye!
```

13. Using Python NLTK, perform the following Natural Language Processing (NLP) tasks for text content.

a) Tokenizing b) Filtering Stop Words c) Stemming d) Part of Speech tagging e) Chunking

f) Named Entity Recognition (NER)

```
import nltk

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk import pos_tag, ne_chunk

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')

text = "Ramesh is working at Google in Bangalore"
tokens = word_tokenize(text)

print("Tokens:", tokens)

stop_words = set(stopwords.words('english'))
filtered = [w for w in tokens if w.lower() not in stop_words]

print("Filtered:", filtered)

ps = PorterStemmer()
stemmed = [ps.stem(w) for w in filtered]

print("Stemmed:", stemmed)

pos = pos_tag(tokens)

print("POS:", pos)

grammar = "NP: {<DT>?<JJ>*<NN>}"
cp = nltk.RegexpParser(grammar)

chunk = cp.parse(pos)

print("Chunk:", chunk)

ner = ne_chunk(pos)

print("NER:", ner)
```

```
Tokens: ['Ramesh', 'is', 'working', 'at', 'Google', 'in', 'Bangalore']
Filtered: ['Ramesh', 'working', 'Google', 'Bangalore']
Stemmed: ['ramesh', 'work', 'googl', 'bangalor']
POS: [('Ramesh', 'NNP'), ('is', 'VBZ'), ('working', 'VBG'), ('at', 'IN'), ('Google', 'NNP'), ('in', 'IN'), ('Bangalore', 'NN
P')]
Chunk: (S
  Ramesh/NNP
  is/VBZ
  working/VBG
  at/IN
  Google/NNP
  in/IN
  Bangalore/NNP)
NER: (S
  (GPE Ramesh/NNP)
  is/VBZ
  working/VBG
  at/IN
  (ORGANIZATION Google/NNP)
  in/IN
  (GPE Bangalore/NNP))
```